디지털논리회로설계

VHDL 과제 2 결과 보고서

담당교수	장범진 교수님
학과	로봇공학과
학번	2023010064
이름	임주언
제출일	2026. 10. 29.

0. 개요

VHDL_1 자료의 Problems 1~5번을 VHDL로 작성하고, Logisim-evolution(Questa 연동)에서 시뮬레이션하여 동작을 확인하였다. 문제마다 Logisim 시트를 하나씩 만들었다(assignment2.circ). 7세그먼트 출력은 수업 예제와 같이 display(7 downto 0) = a b c d e f g dp 순서로 하였다.

1. 문제 1 - AND / OR

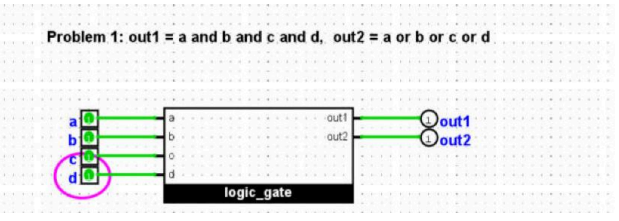
out1 = a and b and c and d, out2 = a or b or c or d 를 그대로 대입문으로 작성하였다. out1은 입력이 모두 1일 때만 1이 되고, out2는 입력 중 하나라도 1이면 1이 된다.

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_arith.all;
use ieee.std_logic_unsigned.all;

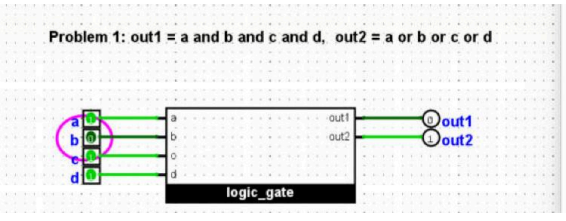
entity logic_gate is
    port(
        a, b, c, d : in  std_logic;
        out1       : out std_logic;
        out2       : out std_logic
    );
end entity logic_gate;

architecture bhv of logic_gate is
begin
    out1 <= a and b and c and d;
    out2 <= a or b or c or d;
end architecture bhv;
```

시뮬레이션 결과



a=b=c=d=1 → out1=1, out2=1



b=0 → out1=0, out2=1

2. 문제 2 - 8-to-1 MUX

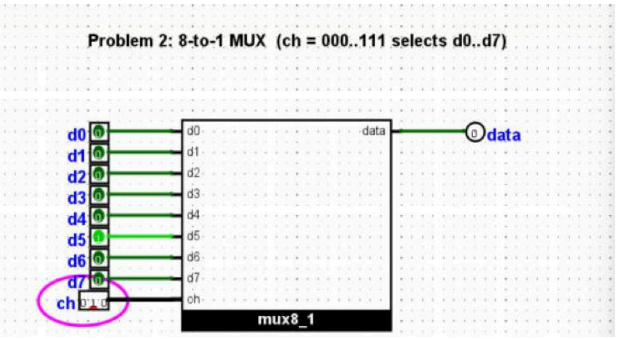
4-to-1 MUX 예제의 case문을 3비트 선택 신호 ch로 늘려서 작성하였다. ch 값에 따라 d0~d7 중 하나가 data로 출력된다. std_logic에는 0, 1 외의 값도 있으므로 when others를 넣었다.

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_arith.all;
use ieee.std_logic_unsigned.all;

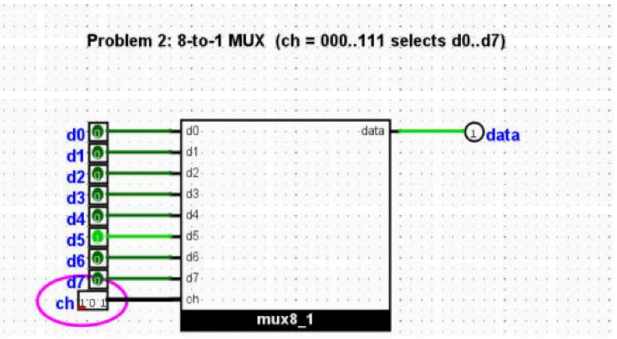
entity mux8_1 is
    port(
        d0, d1, d2, d3, d4, d5, d6, d7 : in  std_logic;
        ch                             : in  std_logic_vector(2 downto 0);
        data                           : out std_logic
    );
end entity mux8_1;

architecture bhv of mux8_1 is
begin
    process(d0, d1, d2, d3, d4, d5, d6, d7, ch)
    begin
        case ch is
            when "000"  => data <= d0;
            when "001"  => data <= d1;
            when "010"  => data <= d2;
            when "011"  => data <= d3;
            when "100"  => data <= d4;
            when "101"  => data <= d5;
            when "110"  => data <= d6;
            when "111"  => data <= d7;
            when others => data <= '0';
        end case;
    end process;
end architecture bhv;
```

시뮬레이션 결과



ch=010 → d2(0) 출력



ch=101 → d5(1) 출력

3. 문제 3 - 7세그먼트 문자 표시

sel(3비트)로 문자를 고르고 en이 0이면 표시를 끄도록 하였다. B와 D는 대문자로 쓰면 숫자 8, 0과 구분이 안 되어서 소문자 b, d로 표시하였다.

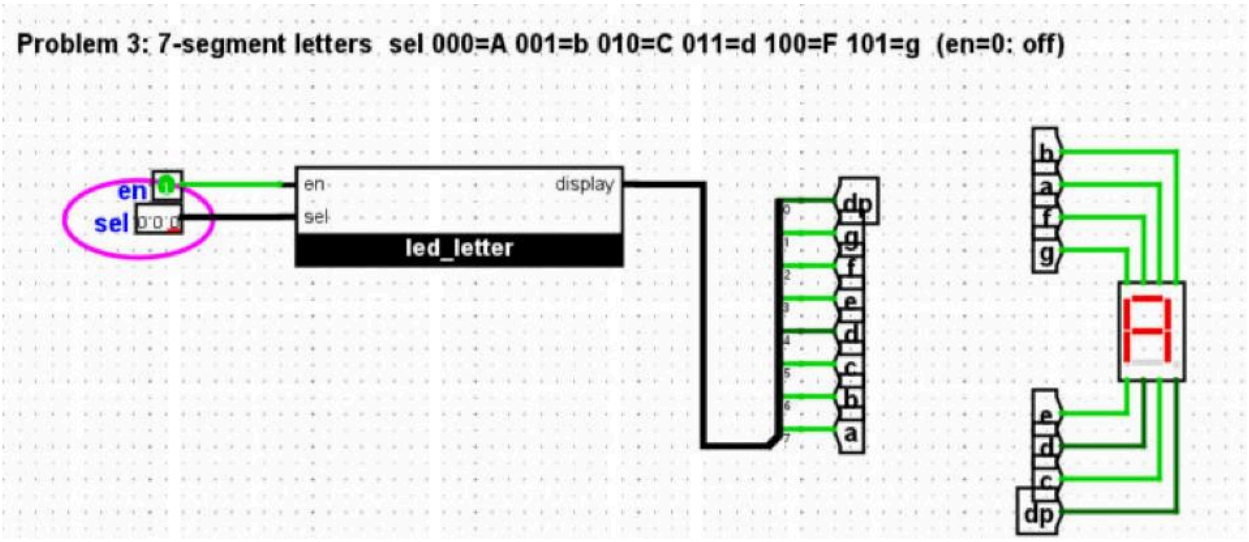
sel	문자	display
000	A	11101110
001	b	00111110
010	C	10011100
011	d	01111010
100	F	10001110
101	g	11110110

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_arith.all;
use ieee.std_logic_unsigned.all;

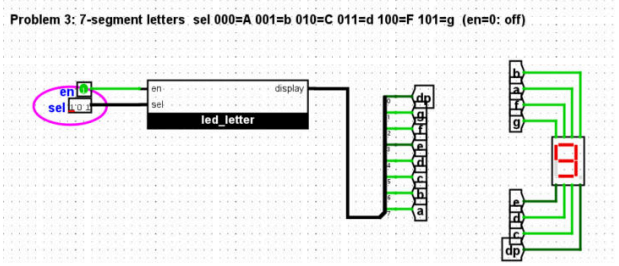
entity led_letter is
    port(
        en      : in  std_logic;
        sel     : in  std_logic_vector(2 downto 0);
        display : out std_logic_vector(7 downto 0)
    );
end entity led_letter;

architecture bhv of led_letter is
begin
    process(en, sel)
    begin
        if en = '0' then
            display <= "00000000";
        else
            case sel is
                when "000"  => display <= "11101110";
                when "001"  => display <= "00111110";
                when "010"  => display <= "10011100";
                when "011"  => display <= "01111010";
                when "100"  => display <= "10001110";
                when "101"  => display <= "11110110";
                when others => display <= "00000000";
            end case;
        end if;
    end process;
end architecture bhv;
```

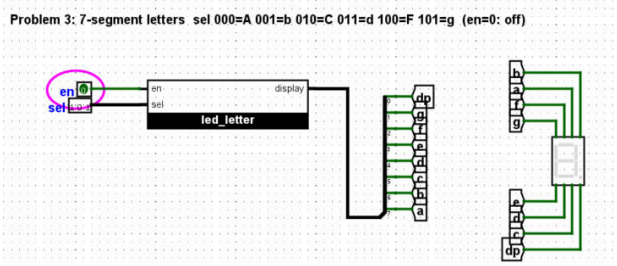
시뮬레이션 결과



en=1, sel=000 → A



en=1, sel=101 → g



en=0 → 꺼짐

4. 문제 4 - 25인 투표기

25명 중 과반, 즉 13명 이상이 1이면 result가 1이 된다. 수업의 13인 투표기처럼 for문으로 1의 개수를 셌다. 다만 예제는 판정이 for문 안에 있어서 입력이 전부 0이면 result에 값이 들어가지 않는 경우가 생기므로, for문이 끝난 후에 한 번만 판정하도록 바꾸었다.

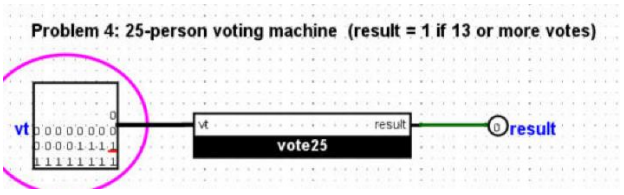
```
library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_arith.all;
use ieee.std_logic_unsigned.all;

entity vote25 is
    port(
        vt     : in  std_logic_vector(25 downto 1);
        result : out std_logic
    );
end entity vote25;

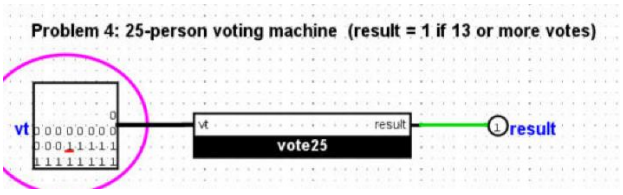
architecture bhv of vote25 is
begin
    process(vt)
        variable sum : integer range 0 to 25;
    begin
        sum := 0;
        for i in 1 to 25 loop
            if vt(i) = '1' then
                sum := sum + 1;
            end if;
        end loop;
```

```
        if sum >= 13 then
            result <= '1';
        else
            result <= '0';
        end if;
    end process;
end architecture bhv;
```

시뮬레이션 결과



12표 → 0



13표 → 1

5. 문제 5 - 1의 개수를 7세그먼트로 표시

문제 4처럼 vt(13비트)의 1의 개수를 세고, 문제 3처럼 case문으로 7세그먼트 값을 출력하였다. 0~9는 숫자, 10~15는 A~F로 표시하고 en이 0이면 꺼지게 하였다. 입력이 13비트라서 실제로는 13(d)까지만 나오지만 E, F도 코드에 넣어 두었다.

```
library ieee;
use ieee.std_logic_1164.all;
use ieee.std_logic_arith.all;
use ieee.std_logic_unsigned.all;

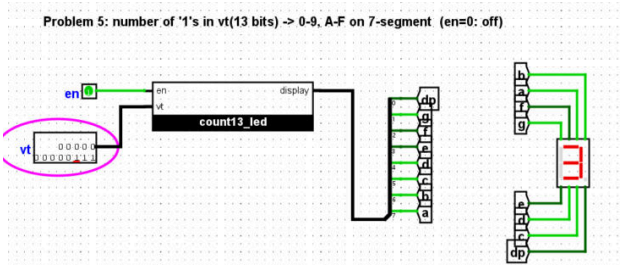
entity count13_led is
    port(
        en      : in  std_logic;
        vt      : in  std_logic_vector(13 downto 1);
        display : out std_logic_vector(7 downto 0)
    );
end entity count13_led;

architecture bhv of count13_led is
begin
    process(en, vt)
        variable sum : integer range 0 to 15;
    begin
        sum := 0;
        for i in 1 to 13 loop
            if vt(i) = '1' then
                sum := sum + 1;
            end if;
        end loop;

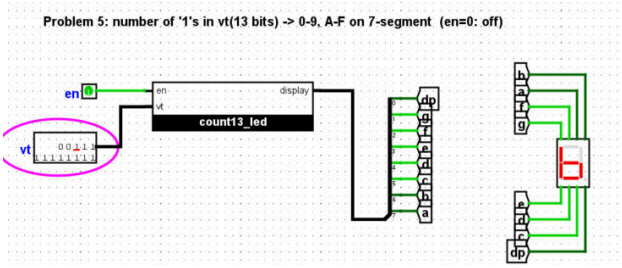
        if en = '0' then
            display <= "00000000";
        else
            case sum is
                when 0      => display <= "11111100";
                when 1      => display <= "01100000";
                when 2      => display <= "11011010";
                when 3      => display <= "11110010";
                when 4      => display <= "01100110";
                when 5      => display <= "10110110";
                when 6      => display <= "10111110";
                when 7      => display <= "11100000";
                when 8      => display <= "11111110";
```

```
                when 9      => display <= "11100110";
                when 10     => display <= "11101110";
                when 11     => display <= "00111110";
                when 12     => display <= "10011100";
                when 13     => display <= "01111010";
                when 14     => display <= "10011110";
                when 15     => display <= "10001110";
                when others => display <= "00000000";
            end case;
        end if;
    end process;
end architecture bhv;
```

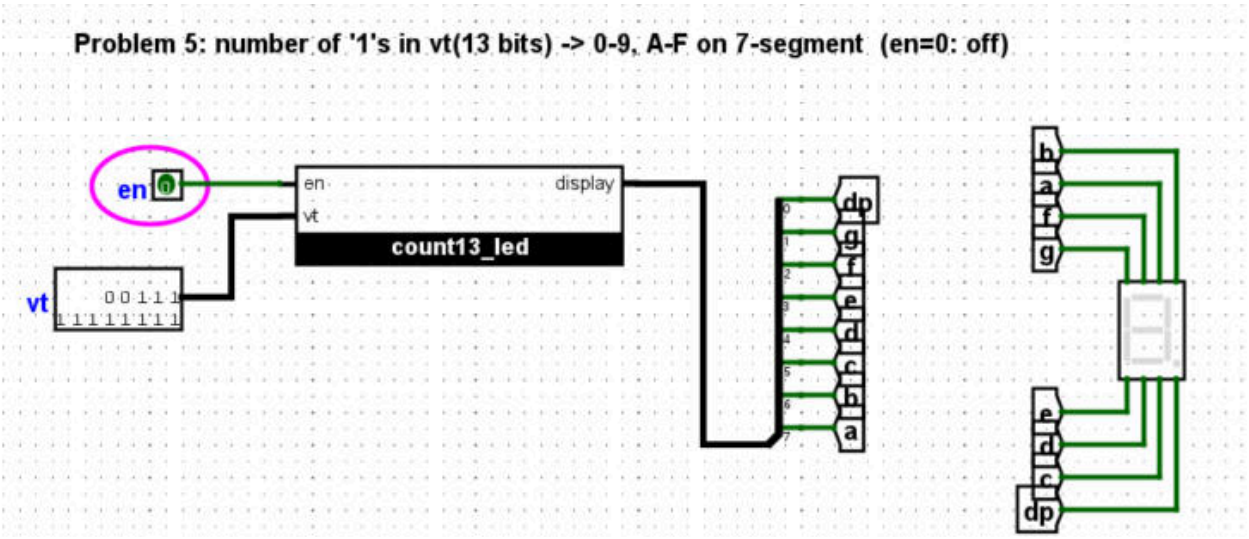
시뮬레이션 결과



1이 3개 → 3



1이 11개 → b



en=0 → 꺼짐

6. 결론

5개 문제 모두 Logisim에서 시뮬레이션하여 원하는 결과가 나오는 것을 확인하였다. MUX와 7세그먼트는 case문, 투표기와 1의 개수 세기는 for문을 사용하면 간단하게 작성할 수 있었다.